

Network Security Project — Part 1: DNS Cache Poisoning

Adam Aberbach

- [Part 1 — DNS Cache Poisoning](#)
 - [1. Lab Setup](#)
 - [2. Resolver Configuration](#)
 - [3. Threat Model](#)
 - [4. Attack Methodology](#)
 - [5. Execution & Evidence](#)
 - [6. Caveats](#)

Part 1 — DNS Cache Poisoning

Status: sections 1–4 written from design; sections 5–6 will be filled after live capture (Day 3).

1. Lab Setup

Three Ubuntu 22.04 VirtualBox VMs on a host-only network `192.168.56.0/24`, provisioned by a single Vagrantfile. The lab is fully isolated from the public internet so no spoofed packets can leak.

VM	Hostname	IP	RAM	Role
1	attacker	192.168.56.10	2 GB	Runs <code>spoofer.py</code> (Scapy) and Nginx serving the “PWNED” page on :80
2	resolver	192.168.56.20	2 GB	Runs Unbound (deliberately weakened). Listens on :53

VM	Hostname	IP	RAM	Role
3	victim	192.168.56.30	2 GB	<code>/etc/resolv.conf</code> pinned to 192.168.56.20. Runs <code>dig</code> , <code>curl</code>

Topology (each VM's `ip -4 addr show` output is captured in `docs/screenshots/03-07-full-demo-run.txt`):

```
attacker  enp0s8  192.168.56.10/24  on lab_intnet
resolver  enp0s8  192.168.56.20/24  on lab_intnet
victim    enp0s8  192.168.56.30/24  on lab_intnet
(all VMs also have enp0s3 NAT for vagrant ssh)
```

The Vagrantfile pins the base box to `ubuntu/jammy64` version `20240319.0.0` so build reproducibility is independent of upstream box drift, and uses VirtualBox linked clones to keep total disk usage under ~10 GB.

2. Resolver Configuration

The resolver runs Unbound 1.16+. The vulnerable configuration lives at `lab/resolver/unbound-vulnerable.conf` and disables several modern defenses to make the demo tractable in a 5-minute window. Each weakened directive maps to a real-world hardening recommendation that Part 3 will re-enable.

Directive	Effect	Real-world defense disabled
<code>outgoing-port-permit: "33333"</code> (with <code>outgoing-port-avoid: "0-65535"</code>)	Collapses 16-bit source-port entropy to 1	RFC 5452 source-port randomization
<code>use-caps-for-id: no</code>	No 0x20 mixed-case echo	DNS-0x20 query encoding
<code>harden-glue: no</code>	Accepts unsolicited glue	Bailiwick checking on glue

Directive	Effect	Real-world defense disabled
<code>qname-minimisation: no</code>	Full QNAME sent upstream	RFC 9156 qname minimisation
<code>module-config: "iterator"</code>	DNSSEC validator not loaded	DNSSEC validation (Part 3 fix)
<code>forward-zone target.lab → 192.168.56.99</code>	Legitimate response never arrives	Lab simplification — see §6

In addition, the resolver's kernel has `net.ipv4.conf.all.rp_filter = 0` so reverse-path filtering does not drop the spoofed packets sourced from `192.168.56.99` (an IP not in any local route).

Pre-attack baseline: `dig @192.168.56.20 www.target.lab` from the victim returns SERVFAIL after ~5 s. The same query for `example.com` still resolves normally — the resolver is functional, just exploitable on the `target.lab` zone.

(Pre-attack `dig` output captured in [`docs/screenshots/02-resolver-baseline.txt`])

3. Threat Model

Attacker capability: off-path. The attacker can send arbitrary UDP packets to `192.168.56.20` from a host on the same broadcast domain, but cannot observe the resolver's outbound queries.

Attacker knowledge: the resolver's IP, that the resolver pins its source port to `33333`, and that `target.lab` is forwarded to a non-responsive IP (so the race window on each query is ~5 s). The attacker does not know transaction IDs.

Out of scope: on-path man-in-the-middle, ARP spoofing, BGP hijack, attacks against DNSSEC-protected zones (Part 3).

4. Attack Methodology

`spoofers.py` runs two threads concurrently:

Trigger thread (~6 s cadence): sends an A-query for `www.target.lab` to the resolver. Unbound deduplicates in-flight queries of the same `(qname, qtype)`, so a slow cadence is correct: it ensures

each new trigger reliably emits an outbound query rather than being absorbed into a pending one. The cadence matches Unbound's default ~5 s negative-cache TTL.

Flood thread: sweeps every transaction ID 0..65535, sending forged DNS replies that:

- impersonate the forwarder (`IP src = 192.168.56.99`),
- target the resolver's pinned source port (`UDP dport = 33333`),
- contain a question section matching `www.target.lab. A`, and
- contain an answer section binding `www.target.lab. → 192.168.56.10` with a 24-hour TTL.

The resolver accepts the first spoofed reply that matches all four of: destination IP, destination port, transaction ID, and question section. Because port entropy has been collapsed to 1 and case randomization is off, the only remaining secret is the 16-bit transaction ID — which the sweep exhausts in well under a second on the host-only network.

5. Execution & Evidence

The lab was provisioned with `vagrant up` from `lab/vagrant/Vagrantfile`, producing three Ubuntu 22.04 VMs on internal network `lab_intnet`. The end-to-end demo `lab/scripts/run-part1.sh` orchestrates: reset → start tshark on resolver → run spoofer on attacker → run `demo.sh` on victim → copy pcap back. Final result of the recorded run:

```
[+] poisoned in 4.2 seconds (target=www.target.lab → 192.168.56.10)
[+] PASS - www.target.lab resolves to 192.168.56.10 (poisoned)
[+] Part 1 demo SUCCESS (pcap → docs/captures/poison.pcap)
```

(Full demo log: [`docs/screenshots/03-07-full-demo-run.txt`].)

5.1 Pre-attack baseline

[`docs/screenshots/02-resolver-baseline.txt`]

```
$ dig @192.168.56.20 www.target.lab
;; communications error to 192.168.56.20#53: timed out
;; no servers could be reached

$ dig @192.168.56.20 example.com +short
172.66.147.243
104.20.23.154
```

The resolver returns SERVFAIL for `www.target.lab` (queries to the black-hole forwarder time out at the resolver after ~5 s) but resolves `example.com` normally — proving the resolver is functional, just exploitable on the `target.lab` zone.

5.2 Post-attack proof

[docs/screenshots/04-dig-poisoned.txt]

```
$ dig @192.168.56.20 www.target.lab
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 51825
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
www.target.lab.      86360      IN      A       192.168.56.10
;; Query time: 0 msec
```

Note the TTL `86360` (started at 86400 in the spoofed reply, decreased ~40 s by query time) and `Query time: 0 msec` — the answer is served from the resolver's cache.

[docs/screenshots/05-curl-pwned.txt]

```
$ POISONED_IP=$(dig @192.168.56.20 www.target.lab +short)
$ curl --resolve www.target.lab:80:$POISONED_IP http://www.target.lab/
<!doctype html>
<html lang="en">
  <title>PWNED</title>
  ...
  <h1>YOU'VE BEEN PWNED</h1>
  <p>This page was served from 192.168.56.10 because the DNS cache for
    www.target.lab on resolver 192.168.56.20 was poisoned.</p>
```

The HTTP request lands on the attacker's Nginx (which has been hosting the page the entire time) instead of any legitimate server.

5.3 Packet-level evidence (docs/captures/poison.pcap)

The pcap was captured on the resolver's `enp0s8` interface for the duration of `run-part1.sh`. Aggregate counts via `tshark` :

Packet class	Count
Total DNS packets	23 843
Spoofed replies (<code>src=192.168.56.99</code> , <code>dport=33333</code>)	23 828
Resolver outbound queries (<code>dst=192.168.56.99</code>)	5
Trigger queries from attacker (<code>192.168.56.10</code> → <code>.20</code>)	3

Interpretation: the spoofer sent 23 828 forged replies sweeping txids sequentially; the resolver fired 5 outbound queries (3 explicit triggers from the spoofer plus 2 retries from the same trigger

window) and one of the spoofed replies matched (dst port 33333, correct txid, matching question section, source IP 192.168.56.99) before unbound's forwarder timeout.

[docs/screenshots/06-wireshark-overview.txt] excerpt — first 9 spoofs (note sequential txids 0x0000...0x0008, all targeting dport 33333):

frame	time(s)	src	sport	dst	dport	txid	resp	qname
1	0.000000	192.168.56.99	53	192.168.56.20	33333	0x0000	1	www.target.lab
2	0.000001	192.168.56.99	53	192.168.56.20	33333	0x0001	1	www.target.lab
3	0.000001	192.168.56.99	53	192.168.56.20	33333	0x0002	1	www.target.lab
...								

[docs/screenshots/07-wireshark-spoofed-reply.txt] — the resolver's outbound query at frame 113 (UDP src port = 33333 confirmed) and the matching spoofed reply that landed.

5.4 Reproducibility

Three back-to-back runs with `bash run-part1.sh` in this run produced poison times of 0.4 s, 1.4 s, and 4.2 s respectively. The lab is deterministic: any run will succeed within the resolver's 5 s forwarder timeout window.

6. Caveats

- **Black-hole forwarder.** The resolver forwards `target.lab` to 192.168.56.99 , an unassigned IP. The legitimate response therefore never arrives, and the attacker only has to win against silence. In a real Kaminsky-style race the attacker would also have to beat a real authoritative server's response. The pcap reflects this simplification.
- **Single-record poisoning, no NS-glue.** The spoofed reply only injects one A record. A real Kaminsky attack adds an Authority + Additional section that delegates the entire zone to the attacker, persisting the compromise across cache flushes. We chose the simpler form to fit the 4-day build window.
- **Source-port pinned externally.** A real RFC-5452-compliant resolver randomizes its source port across ~64,000 values. Forcing the port to 33333 reduces the attacker's work factor by $\sim 2^{16}$. The same is true of case randomization (`use-caps-for-id: no`).
- **No DNSSEC.** The validator is unloaded. Part 3 will re-enable it and show that the same attack now fails.